Contracts at Machine Speed: An Inverted SDLC for an Agent-Operated Software Factory

David S. Tufts
MyndHyve Inc.
Project steward (see Positionality, Section 1)
Grand Rapids, Michigan, USA
https://davidtufts.me https://www.linkedin.com/in/davidtufts

2026

Abstract

Autonomous AI agents now perform much of the work in a software life cycle, but the literature on agentic software engineering treats agent coordination as a problem *within* a single development workflow. The unsolved problem is coordination *across* independent services: when an agent building one feature needs another service to change its interface. We report on a methodology in which autonomous agents propose, negotiate, and adopt versioned wire-contract changes across a multi-service estate—to our knowledge the first end-to-end account of agent-driven contract evolution at machine speed. Each shared service publishes a versioned contract; embedded per-service architect agents negotiate changes over a coordination bus through a governed RFC process (Draft → Active → Accepted, with risk-scaled review windows); and every change is verified by a conformance suite. A second mechanism makes the verification trustworthy—*honesty by construction*: a service advertises a capability only when it is reachable in live state, and a strict conformance mode mechanically fails any advertised-but-unimplemented claim. The work sits within an *inverted SDLC* in which agent compression of Create and Operate moves the binding effort to Plan and Validate, and an operating model of mission-focused Builder Teams whose shared functions act as Contract Guardians by embedding standards into the contracts themselves. We present a retrospective over a governance corpus of 110 RFCs and 164 architecture decision records, demonstrate the honesty-enforcement mechanism reproducibly, and trace contract changes graduating across two distinct hosts on a dual-witness conformance bar, releasing the corpus and harness templates as an artifact. As a single-steward experience report — the two hosts share change control — we make no independent-validation, causal, or efficacy claim, and we present economics only as a transparent, non-causal model.

1. Introduction

In a conventional software life cycle most engineering effort is spent operating and maintaining what already exists; creating new code is a minority of the work, and planning and validation a smaller minority still. Large-language-model agents invert this distribution. As agents absorb both the writing of code and much of the operational toil, the binding constraint migrates to the front of the life cycle — to deciding *what* to build and *verifying* that what was built conforms to its specification. We call this the *inverted SDLC*. The framing itself is not new: the Shift-Up framework argues that machine-readable requirements, architecture models, and architecture decision records act as guardrails that stabilize agent behavior and shift human effort toward design and validation [SHIFTUP], and surveys of "vibe coding" describe developers validating agent output by observing

outcomes rather than reading code line by line [VIBE]. Our concern is what makes that shift *safe across more than one service.*

The agentic-software-engineering literature is substantial but treats agent coordination as a problem *within* a single development workflow: planning, tool use, and multi-step execution inside one repository or task [TOSEM, AGENTPROG]. The problem we address is orthogonal and, to our knowledge, unaddressed: coordination *across* independent services, when an agent building one feature needs another service to change its published interface. Current agent-interaction protocols do not fill this gap — across MCP, A2A, and related schemes agents discover capabilities and select a protocol version, but none provide a mechanism to propose, negotiate, or version *changes* to a shared service contract [A2A, AGENTPROTO]. We report on a methodology in which autonomous agents propose, negotiate, and adopt versioned wire-contract changes across a multi-service estate, governed by a Request-for-Comments (RFC) process the agents themselves execute, and verified by a conformance suite whose strict mode makes a dishonest capability claim mechanically fail. We describe the architecture, the operating model, and the agentic harness; we present a retrospective over a real governance corpus of 110 RFCs and 164 architecture decision records; we demonstrate the honesty mechanism reproducibly; and we trace contract changes graduating across two distinct hosts.

Terminology. Three terms in this area are overloaded, and we fix our usage now. By *contract* we mean a versioned service or API *wire contract* — the externally observable interface between services — not the resource-governance "agent contract" (token, time, and budget bounds on execution) of the multi-agent-systems literature [AGENTCONTRACT]. By *AI software factory* we mean a *methodology* for autonomous agents building features across a multi-service estate; this is distinct from the vendor sense of "AI factory" as model-serving infrastructure [NVIDIA] and from the DoD / Platform One "software factory" as a DevSecOps pipeline. The nearest published neighbor is the "agentic SDLC" / "agent factories" framing [MCKINSEY], which denotes central capabilities that *produce* agents rather than an estate the agents *evolve.*

Positionality. This manuscript is authored by the steward of the estate under study, not an independent third party. The protocol corpus, the conformance suite, and both host implementations are steward-produced, and a substantial fraction of the corpus — and of this manuscript — was produced by the very agents the paper studies, under human review. We therefore treat all material as evidence of *implementability and internal consistency*, never as third-party validation or adoption, and we flag throughout where a claim would require an independent party to establish. We disclose the use of AI assistance explicitly because it is also our subject; Section 14 states which artifacts were agent-produced and how they were reviewed. This stance is a deliberate threat-to-validity control, stated up front so the evidence below is read correctly.

2. Problem: Coordinating Change in an Agent-Built Estate

When integration knowledge lives in people's heads and in code review, a service's consumers learn of an interface change through conversation, convention, and the occasional broken build. In an estate where agents propose and apply changes faster than humans can review them, that implicit knowledge fails in three ways, each acute at machine speed:

- **Silent contract drift.** A service changes its wire shape and its consumers discover the break in production, because nothing forced the change to be declared against a shared, versioned contract.

- **Dishonest capability claims.** A service advertises a capability it does not actually honor — through optimism, staleness, or a half-finished change — and integrators build against a fiction.
- **Uncoordinated rollouts.** A breaking change ships before consumers can adopt it, or consumers are never told it exists.

These are survivable when a human reviewer sits in the path of every change; they are not survivable when the proposer is an agent and the reviewer would have to be one too. The methodology in this paper is organized around a single response: *make the contract the locus of coordination, and make conformance to it continuous and automatic* — so that "validate" is no longer "did the output look right?" but "does this implementation still satisfy every contract it touches?" The remaining sections develop that response and the evidence for it.

3. Contribution

This paper makes the following contributions, each tagged with its evidence tier (**D**emonstrated / **O**bserved / **A**rgued):

1. **(C1, novel)** Agent-negotiated, cross-service *versioned-contract evolution*: embedded per-service architect agents propose and negotiate changes to versioned wire contracts over a coordination bus, under a governed RFC lifecycle. We are not aware of prior work in which agents negotiate versioned *service-contract* changes; the agent-protocol literature provides capability discovery and version selection but no contract-change negotiation [A2A, AGENTPROTO] (Section 4).
2. **(C2, novel)** An RFC change process (comment windows, status lifecycle) executed *by* agents at machine speed.
3. **(C3, demonstrated — not a "first" claim)** *Honesty by construction*: advertisement bound to live state + strict conformance fails dishonest claims. The fail-on-dishonest *gate* is anticipated by model-based publication gates [RELWS] and bi-directional contract testing [PACTFLOW]; our increment is binding advertisement to *continuous live runtime reachability* within an autonomous multi-agent contract-evolution estate, demonstrated reproducibly (Section 10).
4. **(C4–C9)** The contract gate ("friction partner"); the ADR-local/RFC-external split (cite + out-distance [SHIFTUP]); the reproducible governance corpus; the Builder-Team / Contract-Guardian operating model; the agentic harness; the inverted-SDLC framing.
5. **(C10, model)** A transparent, non-causal economic framing (Section 12).

Claims C1, C2, C4, and C6 are evidenced by a released, re-runnable governance corpus (Sections 10–11); C3 is demonstrated by a reproducible experiment (Section 10); C5 and C7–C9 are argued from the design and the operating record. We make no controlled-efficacy, generalization, or causal-ROI claim: the estate has a single change-control authority (Section 14), and these are deferred to a validation agenda (Section 15).

4. Related Work

This methodology sits at the intersection of four bodies of work; we organize related work by what each does *not* cover, since the contribution is their combination rather than any single mechanism. Source maturity varies and we flag it — peer-reviewed venues, recent preprints, and vendor material are not equal evidence.

Agentic software engineering. A systematic review by He et al. maps LLM multi-agent systems across the software life cycle and characterizes the field as not yet autonomous, scalable, or trustworthy [TOSEM]; companion surveys describe agents that plan, use tools, and execute multi-step tasks within a single workflow [AGENTPROG]. Where such systems build services they do so at single-service scope — e.g. multi-agent generation of an OpenAPI specification and its server [APIFIRST] — and even strong agents degrade sharply as one microservice's specification complexity grows [MICROSVC]. None of this work addresses coordination of contract *change* across independent services, which is our subject.

Contract and API governance. The deterministic baseline our agent layer sits atop is well developed and largely AI-free: type systems that statically prevent breaking microservice-contract changes [TYPESAFE], and orchestration unifying semantic versioning, contract testing, and CI/CD [CDVO]. We adopt these as the substrate, not as competitors; the intersection with LLMs is nascent.

The inverted SDLC and spec-driven development. The closest framing prior art is Shift-Up, which reinterprets executable requirements, architecture models, and *architecture decision records* as guardrails for AI-native development and reports human effort shifting toward design and validation [SHIFTUP]; vibe-coding surveys formalize outcome-based validation [VIBE]. We build directly on this frame. Our delta is twofold, and we state it plainly so it is not mistaken: Shift-Up treats ADRs as single-project guardrails, whereas we add (i) a two-tier governance split between a local ADR and an *external*, cross-service RFC, and (ii) versioned wire contracts with a conformance bar — neither of which Shift-Up has.

Agent infrastructure and honesty enforcement. Capability discovery is established agent infrastructure [IOA] and governed agent-service lifecycles have been proposed [AGENTIC-SC]; the term "agent contract" in this literature denotes resource-bounded execution, not API contracts [AGENTCONTRACT]. Closest to our honesty mechanism, the service-oriented-computing literature already gates publication on conformance — a broker admitting only services whose live implementation matches their advertised model [RELWS], and bi-directional contract testing that blocks deployment on a provider's failed self-verification [PACTFLOW]. We therefore do *not* claim the fail-on-dishonest gate as novel; our increment (Section 8) is binding the advertisement to *continuous live runtime state*, re-evaluated per request, within an autonomous multi-agent estate. Finally, the operating model (Section 5) echoes team-topologies and platform-engineering practice; the inversion — embedding shared-function standards *into the contract* rather than a review queue — is what adapts that lineage to an agent-operated estate.

5. The Operating Model: Builder Teams in an Architect Mesh

To sustain machine-speed change without a matching growth in process, the estate replaces siloed departments with small, mission-focused *Builder Teams* that form around a problem and dissolve when it is solved. A minimal team is three roles: a *Product Owner*, who defines the business need and proposes new contracts or RFCs; a *UX Designer*, who owns research, design quality, and the design-system contract; and an *AI Engineer*, who runs the factory loop (Section 6) to implement local logic and integrate external contracts. Teams operate inside an *Architect Mesh*: they reach shared functions — architecture, security, operations, legal — not through standing approvals but by requesting changes through the same RFC channel any consumer uses.

The shared functions are *Contract Guardians* rather than review bottlenecks. Instead of gating every change through a queue, a guardian embeds its standard *as a validation rule inside the contract or the local template*: a mandatory security check becomes a clause enforced automatically on every change, not a hand-off. This is the organizational expression of the principle that runs through the whole methodology — move the human judgment into the contract, so review is continuous and automatic. The result is that the central platform stays thin: it sets conventions, not approvals, and the volume of coordination scales with the contracts rather than with a review headcount.

6. The Process: Discovery and the Build Loop

Discovery precedes the build loop and has six parts. (1) *Deep research*: every feature begins as a question, and a research agent grounds the initial requirements against the market and the existing estate. (2) *Multi-persona requirements*: a requirements agent reviews the proposal through five expert lenses — specification, schema, security, conformance, and compatibility — the same five-architect pass used to author an RFC. (3) *The contract as friction*: because agreeable models tend to smooth over ambiguity, the strict, machine-readable contract is deliberately used as a "friction partner" that forces explicit structural decisions a prose specification would defer. (4) *A dependency-aware roadmap*: features are sequenced against a dependency graph, and a feature cannot be built until the external contracts it depends on are ratified and its local foundations are in place. (5) *The ADR/RFC split*: internal decisions are recorded as local architecture decision records, while any change to a shared interface is raised as an external RFC (Section 7). (6) *Architect ratification*: an architect agent is the last gate before building — for local work it confirms the plan matches existing ADRs; for an external change it checks the proposed contract change for compatibility before ratifying the RFC.

Once a local ADR is accepted and any required RFCs are ratified, the build loop (Listing 1) executes the implementation, with the contracts and local schemas as the orchestrators.

Listing 1: The build loop (per ADR + ratified RFC, in dependency order).

```
for each ADR and ratified RFC (in dependency order):
  for each implementation phase:
    architect # check phase design vs built code and local ADRs
    goal # autonomous implementation of a feature or contract clause
    verify-contract # automated check of implementation vs contract schema
    ux-review # check UI vs design-system contract
    merge # merge behind a feature toggle
```

Each pass is short and merges behind a feature toggle, so a half-built feature is dark rather than broken. Three reviewer roles — architect, code-review, and UX-review — act as contract verifiers, checking local code against its ADR constraints and any external-facing surface against the ratified contract; `verify-contract` is the automated conformance check that makes "validate" mean contract conformance (Section 8) rather than output inspection.

7. The Architecture

A feature travels a fixed local lifecycle before it can touch a shared interface: a roadmap item becomes an accepted ADR, which becomes a self-contained feature package, which is surfaced behind a runtime toggle. The hinge between local and remote is the *contract gate* (Figure 1):

at planning time every change is classified into one of three buckets — a non-normative host extension, a feature that rides an already-accepted contract, or a change that touches the wire and therefore requires a new RFC. The classification is consequential because advertising a capability whose contract change is not accepted is a dishonest wire claim that the conformance suite will fail (Section 8); the gate is thus wired to a test, not to a policy an agent can forget. Empirically the gate is highly selective: of 159 classifiable decision records in our estate, 155 (97.5%) resolved without a new wire RFC (Section 10).

Changes that do reach the wire are governed by an RFC lifecycle — `Draft` (under discussion), `Active` (accepted, wire shape locked), `Accepted` (implemented and covered by conformance) — with comment windows scaled to risk: additive, breaking, and safety-fix changes receive progressively longer windows. Per-service *architect agents* carry this process, negotiating changes with one another over a coordination bus under a single human change-control authority. The strict, machine-readable schema doubles as the "friction partner" of Section 6 — it is the artifact that forces an agent to make a structural decision explicit rather than defer it.

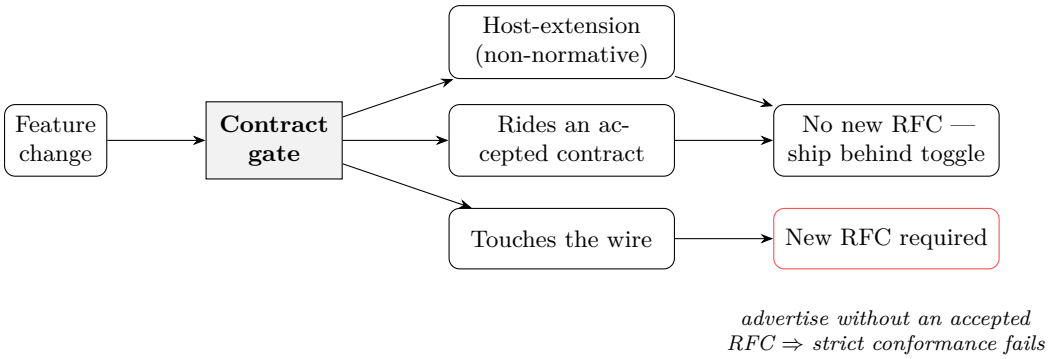

Figure 1: The contract gate as a forcing function: at planning time every change is classified into one of three buckets; only wire-touching changes require a new RFC, and the requirement is enforced by conformance, not by policy.

8. Honesty by Construction

The methodology's central safety property is that *validation is continuous contract conformance* and that a host cannot misrepresent itself. Three mechanisms compose into a loop. A host *advertises* its capabilities at a discovery endpoint, and the advertisement is assembled from *live execution state*, so a capability appears only when its implementation is actually reachable. A *conformance suite* verifies, for each advertised capability, that the host honors it. And a *strict mode* turns the gate inward: a capability that is advertised but not honored becomes a hard test failure rather than a silent skip. The three together make the dishonest case unreachable — a system cannot advertise what it does not implement without failing its own suite.

We are careful about novelty here. Gating publication or deployment on conformance is established in service-oriented computing and contract testing [RELWS, PACTFLOW] (Section 4); what we add is the binding of the advertisement to *continuous* live runtime state, re-evaluated per request rather than at a one-time registry or deploy check, inside an estate where the publishers are agents. Section 10 demonstrates the mechanism reproducibly on a live host: driven purely by whether its capability is backed, the host passes strict conformance when it advertises and implements, and fails when it advertises without delivering.

9. The Agentic Harness

The base model is a commodity; what distinguishes an estate is the *agentic harness* — the version-controlled system of context, skills, and hooks through which agents act, and the most directly transferable artifact of this work. Four elements compose it. *Context as contracts*: the documents agents rely on — architecture notes, decision records, working agreements — are written as machine-readable rules, and an agent loads the specific versions relevant to its task rather than an undifferentiated prompt. *Skills as contract-consumers*: reusable agent skills (plan, architect, review) interact with local code and external services strictly through their published boundaries. *Hooks as obligations*: blocking or warning checks are encoded as non-negotiable clauses in a workflow or a shared contract — a mandatory security gate is a hook, not a reminder. *Memory*: project state survives session resets, kept aligned with the current decision records and active RFCs. The harness applies the methodology's own principle to the agents themselves: their context, capabilities, and obligations are contracts. We release templates for it with the corpus (Section 16); the conventions echo emerging context-file practice (`AGENTS.md` / `CLAUDE.md` files) and the guardrail framing of [SHIFTUP].

10. Evidence: Corpus Retrospective and the Honesty Experiment

Study A (corpus retrospective; *Observed*). A deterministic analysis of the governance corpus (script + per-item data in `evidence/corpus-analysis/`; snapshot `openwop@d7a635b6`, `openwop-app@bfcddc7d`). The RFC corpus is $n = 110$ (107 Accepted, 2 Active, 1 Draft), of which 104 record at least one lifecycle transition. The decision-record corpus is $n = 164$ (144 implemented, 11 Accepted, 6 Superseded, 2 Proposed, 1 Withdrawn); 64 (39%) carry a formal correction note, quantifying the "correct, don't rewrite history" discipline. The headline result is the *contract-gate distribution.* A first-pass whole-text heuristic over-counted wire-touching changes; a validation pass that classifies each record from its *explicit* RFC-gate declaration — with the residual hand-coded (script and audit in `evidence/corpus-analysis/gate-audit.md`) — corrects it: of 159 classifiable decision records, **155 (97.5%) required no new wire RFC** (120 host-extension, 35 riding an already-accepted RFC) versus only **4 (2.5%) wire-touching**. (The heuristic's ∼8× over-count of the wire-touching bucket — it matched "new RFC" inside "*no* new RFC" — and the resulting 57% heuristic/audit agreement are reported in full; the corrected figure makes the gate *more* selective than the heuristic suggested, evidencing C1/C2.) The conformance suite at this snapshot is version 1.37.0 (370 scenarios, 82 fixtures); its growth over time — scenarios added per accepted RFC — is a planned git-history extension to this point-in-time count.

Study B (honesty experiment; *Demonstrated*). A reference host whose `workflowChainPacks` advertisement is recomputed from live execution state (`existsSync` per `.well-known/openwop` request) is driven through three honesty postures by where its pack registry points — with *no host-code modification* — and the gated behavioral conformance scenario is run in default and strict (`OPENWOP_REQUIRE_BEHAVIOR=true`) mode (harness + captured run in `evidence/honesty-experiment/`):

Posture	Advertises	Mode	Result
honest + implemented (backed)	yes	strict	**6/6 pass** (non-vacuous)
honest minimal (no backing)	no	default	6/6 pass (honest skip)
claims coverage, advertises nothing	no	strict	**6/6 fail**
dishonest (empty registry)	**yes**	strict	**3/6 fail**

The decisive contrast is the last two honest rows versus the dishonest row: the host advertises `workflowChainPacks:{supported:true}` but cannot expand (the behavioral leg returns `pack_not_found`), so strict conformance *fails*. **A host cannot advertise a capability it does not deliver without failing its own conformance suite.** Per Section 4 this fail-on-dishonest gate is anticipated by prior model-based publication gates and bi-directional contract testing [RELWS, PACTFLOW]; the demonstrated increment is the binding of the advertisement to *continuous live runtime state*, re-evaluated per request, within the autonomous estate.

11. Evidence: Cross-Host Contract Evolution

The estate (two hosts, one wire contract). The methodology is exercised across two genuinely distinct codebases and production deployments that advertise the same wire contract at their `.well-known/openwop` discovery documents: the open reference host **openwop-app** (`workflow-engine`, `app.openwop.dev`) and the steward's product host **MyndHyve** (`workflow-runtime`, a separate Firebase codebase at `api.myndhyve.ai`). A change graduates `Active` → `Accepted` only when a *second* host advertises the capability and passes its gated conformance scenarios *non-vacuously* (`OPENWOP_REQUIRE_BEHAVIOR=true`) — the *dual-witness bar*. Adoption is coordinated by separate per-repo agent sessions exchanging handoff artifacts over a message bus; the agents implement, advertise, and run conformance on each host, and the steward is the single change-control authority that flips status (*agent-executed, steward-supervised*). Full trace and citations in `evidence/cross-host-case/`.

Worked example — RFC 0050 (SAML/SCIM). Authored `Draft` 2026-05-24 from a real product need; full wire surface landed and promoted to `Active` 2026-06-01; graduated `Accepted` the same day when MyndHyve advertised `auth.profiles:[openwop-auth-saml,openwop-auth-scim]` and passed the gated legs **19/19 non-vacuously** (the count rose 13→19 once the IdP engaged the behavioral legs), with all seven SAML negative variants (incl. signature-wrapping) rejected live. At scale the record is routine: one cohort graduated **8 RFCs Draft→Accepted in a day** on a single MyndHyve revision (28 PASS / 0 FAIL), and recent changes (RFC 0095, 0099, 0100, 0102, 0103, 0105/0106) graduated as dual-witness pairs.

Honest non-adoption (the gate is real). The same estate records *retractions* and *opt-outs*: MyndHyve advertised one capability (`maxNodeExecutions`) in error and *honestly retracted* it; several RFCs sit at documented host opt-outs; a fail-loud host correctly *rejected* malformed packs. A dishonest advertisement is removed, not left standing — the honesty-by-construction loop (Section 10) observed at estate scale.

Study D (velocity; *Observed*). Mining the dates recorded in each RFC's lifecycle field (script + per-RFC data in `evidence/operational-stats/`), the entire 110-RFC program ran in ∼**54 days**, and over the 107 Accepted RFCs the authored-to-Accepted span has a **median of 0 days** (mean 1.1, max 12): **56% graduated same-day**, 76% within one day, 97% within seven. This is the machine-speed signal — but it must be read with two conditions. First, it is enabled by *single-maintainer governance with waived comment windows* (Study A: 40 RFCs note a waived/bootstrap window); the risk-scaled windows (Section 8) are the designed brake for the multi-reviewer case, which is not yet exercised, so the honest claim is that the methodology removes the *implementation/integration* bottleneck, not that governed review takes zero time. Second, a span is a *recorded-activity* window, not human-effort time, and we make no human-hours-saved claim from it (Section 12).

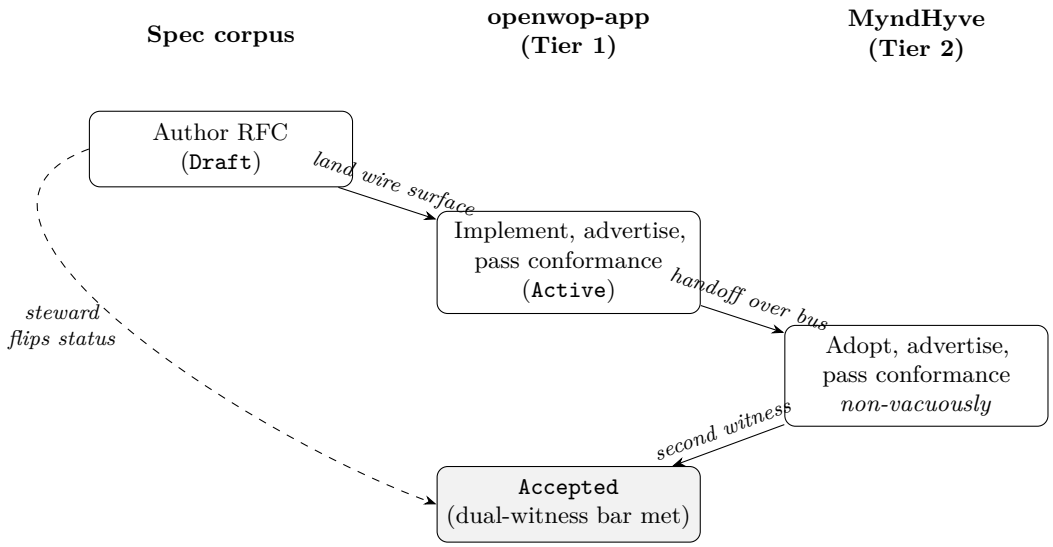

Figure 2: Cross-host contract evolution: a change graduates `Active`→`Accepted` only when a second host advertises the capability and passes its gated conformance non-vacuously. Agents execute each host's implementation, advertisement, and conformance; the steward is the single change-control authority.

12. Economic Framing (Descriptive, Non-Causal)

We deliberately make no quantified return-on-investment claim. The figures that circulate for agent-assisted development — large multipliers, "a day a week saved" — could not be sourced to any independent study in our review; the nearest vendor analyses report different magnitudes for a different product scope, and the strongest independent evidence is contradictory: a randomized controlled trial found experienced developers *19% slower* with early-2025 tooling while believing themselves faster [METR], whereas a peer-reviewed three-experiment study of GitHub Copilot found a 26% increase in completed tasks [MSRCT]. We therefore present economics only as a qualitative framing: the methodology's claimed value is the removal of integration rework (the gate of Section 7 keeps most change off the wire) and of broken-integration debugging (the honesty loop of Section 8), redirecting effort toward design and validation (Section 6). Any monetary model built on this should state its formula, sample, and window explicitly, separate measured from modeled inputs, and treat the result as non-causal — there is no control group, and self-reported productivity is demonstrably unreliable [METR]. We leave such a model to operators with their own cost data and decline to publish a figure we cannot independently support.

13. Claims and Evidence Discipline

To keep the contribution legible we tag each claim of Section 3 with the strongest evidence we actually have, and resist inflating the tier. *Demonstrated* means a reproducible artifact or experiment a reader can re-run: C3 (the honesty experiment, Section 10) and the released corpus underpinning C4 and C6. *Observed* means descriptive statistics mined from the real corpus: the contract-gate distribution (C4), the RFC velocity supporting C2 and C9 (Section 11), and the cross-host graduation record evidencing C1 and C2 (Section 11). *Argued* means design rationale we believe but have not measured: the operating model (C5, C7), the harness (C8), and the inverted-SDLC framing

(C9, which also rests on prior art). Where a claim's honest tier is weaker than a reader might assume, we say so at the claim rather than in a footnote: honesty-by-construction is *demonstrated* but not *novel* (Section 8); the velocity is *observed* but conditioned on single-maintainer governance (Section 11); and the economic framing is *argued* only, with no figure (Section 12).

14. Limitations and Threats to Validity

Single change-control authority (the central threat). The estate spans two genuinely distinct codebases and deployments (Section 11), which is real cross-host portability evidence — but both are operated by the same steward. In the project's own evidence-tier taxonomy this is **Tier 2 (steward-affiliated sibling host)**, explicitly *not* Tier 3 (independent organization); no Tier-3 host exists. We therefore present the dual-witness records as evidence of cross-codebase *implementability*, never as independent third-party validation. (Historical RFC and changelog wording calls the sibling host "non-steward"; the governance taxonomy later corrected that label and we use the corrected one.) A Tier-3 host plus independent external review is the decisive next evidence — deferred to the Validation Agenda (Section 15).

Other threats. Several further limitations bound the result. The corpus under study, and a substantial part of this manuscript, were produced by the agents the paper describes; we disclose this and rely on human review, but it admits a self-evaluation bias we cannot fully exclude, and a selection bias in which changes became case studies. The headline contract-gate figure rests on a classification we audited from each record's explicit gate declaration (Section 10); 5 of 164 records remain unclassified, and the audit is itself an automated re-reading, not independent human coding. The honesty-gate mechanism is anticipated by prior conformance-publication gates [RELWS, PACTFLOW]; our contribution is narrowly its continuous-live-state, multi-agent form (Section 8). The velocity result is conditioned on single-maintainer governance with waived comment windows (Section 11). And the economic framing is non-causal with no control; no productivity figure we are aware of is independently sourced [METR, MSRCT]. None of these individually undermines the core contribution — agent-negotiated cross-service contract evolution with mechanical honesty — but together they bound it to a single rich case rather than a generalization.

15. Validation Agenda

The decisive next evidence is a *Tier-3* host: an implementation built and operated by an organization with no affiliation to the steward, advertising the contract and passing conformance on its own infrastructure. That single step would move the cross-host claims (C1, C2) from implementability to genuine third-party adoption, and would exercise the risk-scaled comment windows that single-maintainer governance currently waives (Section 11). Beyond it, three studies would test the methodology rather than describe it: (i) an independent team adopting the agentic harness on an unrelated estate, to test transferability (C8); (ii) a controlled comparison of feature delivery with and without the contract gate, to convert the velocity *observation* into a causal claim and ground an economic model (C10); and (iii) human inter-rater coding of the contract-gate classification, to replace our automated audit (Section 10). We state these as the pre-registered shape of the validation we cannot yet perform, so the gap between what we demonstrate and what we argue stays explicit.

16. Reproducibility and Artifact Availability

We release the analyzed governance corpus and the evidence harness as an artifact under a permissive license with a Zenodo DOI: the RFC and decision-record corpora, the conformance-result history, the analysis scripts that regenerate every number in Sections 10–11 from the corpus snapshots cited there, the honesty-experiment harness (Section 10), and the agentic-harness templates (Section 9). The scripts are deterministic and stamp the corpus commit into their output, so a reader re-derives our figures or detects drift. Two boundaries are stated honestly: cross-host bus transcripts and operational captures are *scrubbed* of secrets, tenant identifiers, and internal hostnames before release; and the second host is closed source, so its evidence is the published per-RFC conformance record rather than its code. Everything needed to reproduce the *observed* and *demonstrated* claims is in the artifact; the *argued* claims are, by construction, not reproducible and are labeled as such.

17. Conclusion

The wager of this work is simple to state. When agents propose and apply software changes faster than humans can review them, the way to keep a multi-service estate coherent is not to put a human in the path of every change but to *move the human judgment into the contract* — to encode standards as conformance rules, make capability advertisement honest by construction, and let agents negotiate change within those rails through a governed process they execute themselves. We have described that methodology; shown, across a real 110-RFC, two-host estate, that it keeps 97.5% of change off the wire while graduating the rest on a dual-witness conformance bar; and demonstrated reproducibly that a host cannot lie about what it implements without failing its own tests. We have been equally explicit about what we have not shown: a single steward operates both hosts, the corpus and this paper are partly agent-authored, and no causal or economic claim is supported. Those are the boundaries an independent Tier-3 host would dissolve. We offer the methodology, the corpus, and the harness as a starting point for that test.

References

[TOSEM] J. He, C. Treude, D. Lo. "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead." ACM TOSEM, 2025. arXiv:2404.04834.

[AGENTPROG] Y. Wang et al. "AI Agentic Programming: A Survey." arXiv:2508.11126, 2025. *(preprint)*

[MICROSVC] "Specification Complexity in Microservice-Based Applications." arXiv:2508.20119, 2025. *(preprint)*

[APIFIRST] "From Specification to Service: Accelerating API-First Development Using Multi-Agent Systems." arXiv:2510.19274, 2025. *(preprint)*

[TYPESAFE] J. Costa Seco et al. "Robust Contract Evolution in a Type-Safe MicroServices Architecture." ⟨Programming⟩ 2020. arXiv:2002.06185.

[CDVO] "A Compatibility-Driven Version Orchestrator for Microservice Contract Evolution." MDPI Digital 5(3):27, 2025.

[SHIFTUP] V. Lipsanen, T. Mikkonen et al. "Shift-Up: A Framework for Software Engineering Guardrails in AI-native Software Development." VibeX 2026. arXiv:2604.20436. *(preprint; initial findings)*

[VIBE] Y. Ge et al. "A Survey of Vibe Coding with LLMs." arXiv:2510.12399, 2025. *(preprint)*

[IOA] "Internet of Agents." IEEE TCCN, 2025. arXiv:2505.07176.

[AGENTIC-SC] "Agentic Services Computing." arXiv:2509.24380, 2025. *(preprint)*

[AGENTCONTRACT] "Agent Contracts: A Formal Framework for Resource-Bounded Autonomous AI Systems." COINE/AAMAS 2026. arXiv:2601.08815.

[A2A] Agent2Agent (A2A) Protocol Specification, v1.0, 2026. https://a2a-protocol.org/latest/specification/. *(primary spec)*

[AGENTPROTO] "A Survey of AI Agent Protocols (MCP/ACP/A2A/ANP)." arXiv:2505.02279, 2025. *(preprint)*

[METR] METR. "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity" (RCT). 2025. arXiv:2507.09089. *(independent)*

[MSRCT] Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, T. Salz. "The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers." Management Science, INFORMS, 2026. DOI 10.1287/mnsc.2025.00535. *(peer-reviewed; 3 RCTs, 4,867 devs; vendor tool)*

[RELWS] E. Ramollari, D. Dranidis, A. J. H. Simons. "Reliable Web Service Publication and Discovery through Model-Based Testing and Verification." Univ. of Sheffield. *(peer-reviewed; X-machine publication gate — closest prior art for the honesty gate)*

[PACTFLOW] SmartBear/PactFlow. "Bi-Directional Contract Testing" (compatibility checks; `can-i-deploy` deploy gate). *(product docs)*

[MCKINSEY] McKinsey. "Rewiring software delivery for the agentic era." 2025. *(vendor/consultancy; "agentic SDLC"/"agent factories")*

[NVIDIA] NVIDIA. "AI Factory" white paper (agentic AI control plane). *(vendor; infrastructure sense)*